SPECIFICATION

Calypso Terminal API

Calypso Crypto Asymmetric

Version 0.3.0-SNAPSHOT, 2026-07-20



Warning. The present document cancels and replaces the previous release.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Link and Contact



Website
https://calypsonet.org



Support
support@calypsonet.org

Copyright © Calypso Networks Association, https://calypsonet.org.

This specification is licensed under the **Creative Commons Attribution-NoDerivatives 4.0 International** *(CC BY-ND 4.0).*

You are free to **share** *— copy and redistribute this document in any medium or format for any purpose, even commercially — under the following terms:*

- ***Attribution*** *— You MUST give appropriate credit to the Calypso Networks Association, provide a link to the license, and indicate if changes were made. You MAY do so in any reasonable manner, but not in any way that suggests the Calypso Networks Association endorses you or your use.*
- ***NoDerivatives*** *— If you remix, transform, or build upon this document, you MAY NOT distribute the modified material.*

No additional restrictions — You MAY NOT apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Implementation grant *— The* **NoDerivatives** *condition above applies to this specification* **document** *only — its text, tables and diagrams. It does* **not** *restrict the use of the technical information the document contains. The Calypso Networks Association hereby grants everyone an irrevocable, worldwide, royalty-free permission to use the information contained in this specification to design, develop, and distribute software implementations of this API, in any programming language, under the license of their choice.*

The full license text is available at https://creativecommons.org/licenses/by-nd/4.0/.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Table of Contents

1. Abstract . . . 5
2. Introduction . . . 6
2.1. Purpose . . . 6
2.2. Scope . . . 6
2.3. Conformance . . . 6
2.4. Document Conventions . . . 7
2.4.1. Naming . . . 7
2.4.2. Data types . . . 7
2.4.3. Type tables . . . 7
2.4.4. Operation tables . . . 8
2.4.5. Operation contracts . . . 8
2.4.6. Concurrency . . . 8
2.5. References and Resources . . . 9
2.5.1. Calypso References . . . 9
2.5.2. Normative References . . . 9
2.5.3. External Resources . . . 9
3. Glossary and Acronyms . . . 10
3.1. Glossary . . . 10
3.2. Acronyms . . . 10
4. Architectural Overview . . . 11
4.1. Functional positioning . . . 11
4.2. Logical namespaces . . . 11
4.3. UML class diagram . . . 11
5. API Specification . . . 13
5.1. Constants . . . 13
5.1.1. Asymmetric Crypto API Properties . . . 13
5.2. Data . . . 13
5.2.1. CA Certificate Content . . . 13
5.3. SPI Interfaces . . . 14
5.3.1. Asymmetric Crypto Card Transaction Manager Factory SPI . . . 14
Create Card Transaction Manager . . . 14
5.3.2. Asymmetric Crypto Card Transaction Manager SPI . . . 14
Init Terminal PKI Session . . . 14
Is Card PKI Session Valid . . . 15
Update Terminal PKI Session . . . 15
5.3.3. CA Certificate Parser SPI . . . 15
Get Certificate Type . . . 15
Parse Certificate . . . 16
5.3.4. CA Certificate SPI . . . 16
Check Certificate And Get Content . . . 16
Get Issuer Public Key Reference . . . 16
5.3.5. Card Certificate Parser SPI . . . 17
Get Certificate Type . . . 17
Parse Certificate . . . 17
5.3.6. Card Certificate SPI . . . 17
Check Certificate And Get Public Key . . . 17
Get Card AID . . . 18

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Get Card Serial Number 18
Get Issuer Public Key Reference 18
5.3.7. PCA Certificate SPI 18
Check Certificate And Get Content 19
5.4. Errors 19
5.4.1. Asymmetric Crypto 19
5.4.2. Certificate Validation 19

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

1. Abstract

This document is the official technical specification of the **Terminal Calypso Crypto Asymmetric API** standardised by the **Calypso Networks Association** (CNA). It defines, in a language-agnostic way (using a notation inspired by the Kotlin language), the interfaces, classes, enumerations, errors, structural relationships and behavioural requirements that any conforming implementation of the Terminal Calypso Crypto Asymmetric API MUST satisfy.

The Terminal Calypso Crypto Asymmetric API is the SPI through which the **Terminal Calypso Card API** delegates every asymmetric-key cryptographic operation — in particular PKI-mode secure session computations and the validation of the certification chain associated with PKI-enabled Calypso cards.

Document Status

Reference	YYMMDD-SP-CNATerminalAPI-CalypsoCryptoAsymmetric
Short name	CNA-TCCA-API
Version	0.3.0-SNAPSHOT
Revision date	2026-07-20
Editor	Calypso Networks Association
Source repository	https://github.com/calypsonet/calypsonet-terminal-calypso-crypto-asymmetric-uml-api
Reference license	Creative Commons Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0)



The present document specifies the 0.3.0-SNAPSHOT of the Terminal Calypso Crypto Asymmetric API. It defines a single, coherent baseline against which conforming implementations are evaluated; the **Since** column indicates the version in which each member was introduced.

Revision List



This section lists the high-level changes per version. The complete, fine-grained changelog is maintained in the `CHANGELOG.md` file at the root of the source repository.

Version / Date	Modifications
v0.3.0-SNAPSHOT 2026-07-20	Baseline.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

2. Introduction

2.1. Purpose

The Terminal Calypso Crypto Asymmetric API defines the contract that an **asymmetric-key crypto module** MUST expose so that a Calypso transaction manager can:

- parse, validate and chain Primary CA (PCA), CA and card certificates produced by the Calypso PKI;
- operate a PKI-mode secure session against a Calypso card (init, update, verify card signature).

The contract is intentionally agnostic to the underlying cryptographic library: any component capable of producing the required signatures (as specified in ISO/IEC 9796) and certificate validations MUST be acceptable.

2.2. Scope

This specification covers:

- the data carrier used to expose certificate content to the upstream layer,
- the SPIs implemented by the crypto module to parse and validate certificates,
- the SPI implemented by the crypto module to drive a PKI secure session,
- the factory used by the upstream layer to instantiate the transaction-manager SPI,
- the errors raised by the SPI.

The following topics are **out of scope**:

- the on-the-wire format of certificates beyond their parsing entry point,
- the management of certificate stores and revocation,
- the cryptographic algorithms themselves.

2.3. Conformance

A software product conforms to this specification if and only if:

1. it implements every interface defined in Chapter 5 with the operations and semantics prescribed in this document,
2. it raises errors exclusively of the types defined in Section 5.4 for the situations described,
3. it preserves the structural relationships described in Chapter 4,
4. every object it provides implements the marker interface of CNA-TCC-API that corresponds to its type, as described below.

The Calypso Card API (CNA-TCC-API) declares marker interfaces through which the objects of an asymmetric crypto module are handed over to the card API. Those markers declare no operation: their content is the one defined by this specification. Every object implementing a type of the right column MUST therefore also implement the marker of the left column.

CNA-TCC-API marker	CNA-TCCA-API type
`AsymmetricCryptoCardTransactionManagerFactory`	`AsymmetricCryptoCardTransactionManagerFactorySpi`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

CNA-TCC-API marker	CNA-TCCA-API type
`CaCertificate`	`CaCertificateSpi`
`CaCertificateParser`	`CaCertificateParserSpi`
`CardCertificate`	`CardCertificateSpi`
`CardCertificateParser`	`CardCertificateParserSpi`
`PcaCertificate`	`PcaCertificateSpi`

Throughout this specification, the key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

In conformance with RFC 2119, **SHALL** is equivalent to **MUST**; this specification uses **MUST** exclusively in order to avoid ambiguity.

2.4. Document Conventions

2.4.1. Naming

Type names follow `UpperCamelCase`; operation, parameter and enumeration-member names follow `lowerCamelCase` except for enumeration values which use `UPPER_SNAKE_CASE`. Names defined by this specification are reproduced as-is in the UML class diagram (Section 4.3).

2.4.2. Data types

The following type names, whose notation is inspired by the Kotlin language, are used in operation signatures and are mapped, in each binding, to the closest equivalent native type. Implementations MUST preserve the semantic intent rather than the syntactic form.

Type	Description	Typical bindings
`String`	Sequence of characters, UTF-8 encoded by default.	`String`, `string`, `str`.
`Boolean`	Two-state truth value.	`boolean`, `bool`.
`Byte`	Signed 8-bit value.	`byte`, `i8`.
`Long`	Signed 64-bit integer.	`long`, `Int64`, `i64`.
`ByteArray`	Ordered sequence of octets.	`byte[]`, `[u8]`, `bytes`.
`PublicKey`	Opaque, immutable handle representing an asymmetric public key.	`PublicKey` (Java), `X509PublicKey`, `SubjectPublicKeyInfo`, equivalent.
`Any`	Value of any type, root of the type hierarchy.	`Object`, `Any`, `object`, `Box<dyn Any>`.
`T?`	Nullable value: either a `T` value or `null`.	`Optional<T>`, `T?`, `Option<T>`.
`null`	Absence of value, the alternative held by a nullable type.	`null`, `nil`, `None`.
`Unit`	Indicates that an operation returns no value.	`void`, `Unit`, `()`.

2.4.3. Type tables

Each interface, class, enumeration and error defined in this document is described by a table of the form:

Name	The type's normalized name.
Kind	The category of the type (interface, class, enumeration, error, etc.).

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Namespace	The namespace in which the type is defined.
Extends	The type extended by this type, when applicable.
Implements	The interface implemented by this type, when applicable.
Since	The version of the API in which the type was introduced.
Purpose	A normative description of the type's responsibility.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

The **Purpose** row states, in one or two sentences, the responsibility of the type and the way an instance is obtained. It is meant to be scanned, not read: any content that requires structure or depth—rules, lists, notes, value tables or examples—is placed as prose below the table.

2.4.4. Operation tables

Each operation defined in this document is described by a table of the form:

Signature	The operation's signature.
Since	The version of the API in which the operation was introduced.
Description	A normative description of the operation's behaviour.
Parameters	A description of each input parameter.
Returns	A description of the returned value, if any.
Pre-conditions	The conditions the caller MUST satisfy before invoking the operation, each prefixed by its nature.
Errors	A list of error conditions a correct caller must handle, each mapped to a specific error type.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

2.4.5. Operation contracts

To avoid repetition, the following rules apply to every operation table in Chapter 5 and are therefore not restated for each operation:

- **Pre-conditions.** The **Pre-conditions** row states what the caller MUST guarantee before invoking the operation. Violating one is a contract error, never listed in the **Errors** row, which describes only the outcomes a correct caller must handle. Each entry is prefixed by its nature, which determines the error raised when the condition is not met:
  - **Argument**—an illegal argument error. Applies to any invalid argument, including values bounded by a card or SAM protocol.
  - **Range**—an index out of bounds error. Applies when the argument designates a position within a collection or memory image exposed by the API.
  - **State**—an illegal state error.
  - **Capability**—an unsupported operation error.
- **Non-`null` results.** Unless the return type is marked nullable (`T?`) or the **Returns** row states otherwise, an operation returns a non-`null` value.

2.4.6. Concurrency

Unless explicitly stated otherwise, the stateful types defined by this specification are **not** thread-safe. A given instance MUST be accessed from a single thread at a time; concurrent use of the same instance without external synchronisation results in undefined behaviour. Distinct instances MAY be used concurrently.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

2.5. References and Resources

2.5.1. Calypso References

References to CNA Terminal APIs designate the indicated **major** version; any later release within the same major is backward-compatible per that API's evolution policy.

Reference	Document
[CNA-TCC-API]	*CNA Terminal API — Calypso Card* (SP-CNATerminalAPI-CalypsoCard), version 3.0, Calypso Networks Association.

2.5.2. Normative References

Reference	Document
[RFC 2119]	*Key words for use in RFCs to Indicate Requirement Levels*, S. Bradner, March 1997.
[RFC 8174]	*Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, B. Leiba, May 2017.
[ISO/IEC 9796]	*Information technology — Security techniques — Digital signature schemes giving message recovery*.

2.5.3. External Resources

Resource	Link
Calypso Networks Association	https://calypsonet.org
Terminal APIs website	https://terminal-api.calypsonet.org
Terminal APIs documentation	https://docs.terminal-api.calypsonet.org

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

3. Glossary and Acronyms

3.1. Glossary

Term	Definition
PCA	Primary Certification Authority — the trust anchor of the Calypso PKI certificate hierarchy. Self-signed.
CA	Certification Authority — intermediate authority signed by the PCA or another CA, used to authenticate further CAs or cards.
Card Certificate	Certificate signed by a CA, binding a public key to the card identity.
PKI Mode	Calypso secure-session mode in which authentication of the card relies on asymmetric cryptography.
Public Key Reference	Stable identifier (typically the public key hash) used to look up the issuer's public key.
SPI	Service Provider Interface — an interface designed to be implemented by an extension module.

3.2. Acronyms

Abbreviation	Expansion
AID	Application IDentifier
APDU	Application Protocol Data Unit
CA	Certification Authority
CNA	Calypso Networks Association
PCA	Primary Certification Authority
PKI	Public Key Infrastructure
SPI	Service Provider Interface
UML	Unified Modelling Language

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

4. Architectural Overview

4.1. Functional positioning

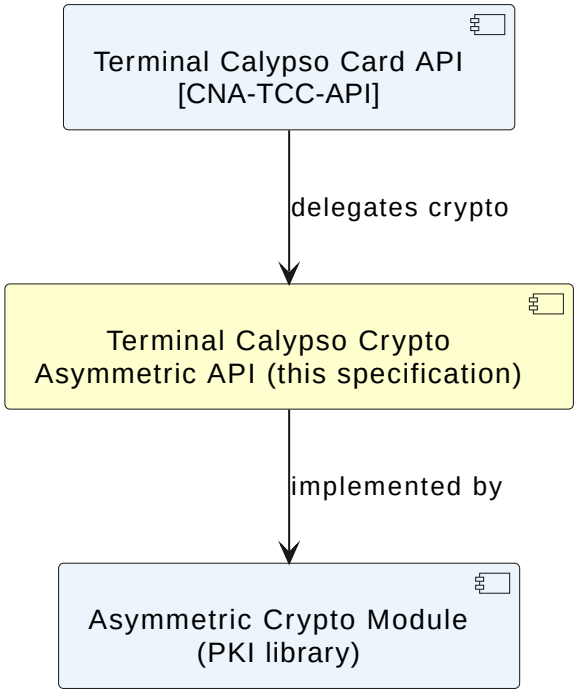


Figure 1. Terminal Calypso Crypto Asymmetric API — Architecture Overview

4.2. Logical namespaces

Namespace	Role	Summary
`calypso.crypto.asymmetric`	Public API	Properties and root errors.
`calypso.crypto.asymmetric.certificate`	Public API	Certificate-validation error type.
`calypso.crypto.asymmetric.certificate.spi`	SPI	Interfaces implemented by the crypto module for certificate parsing and validation, and the certificate content they expose.
`calypso.crypto.asymmetric.transaction.spi`	SPI	Interfaces implemented by the crypto module to drive a PKI-mode card transaction.

4.3. UML class diagram

The following UML class diagram provides a visual representation of the types and relationships defined by this specification:

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

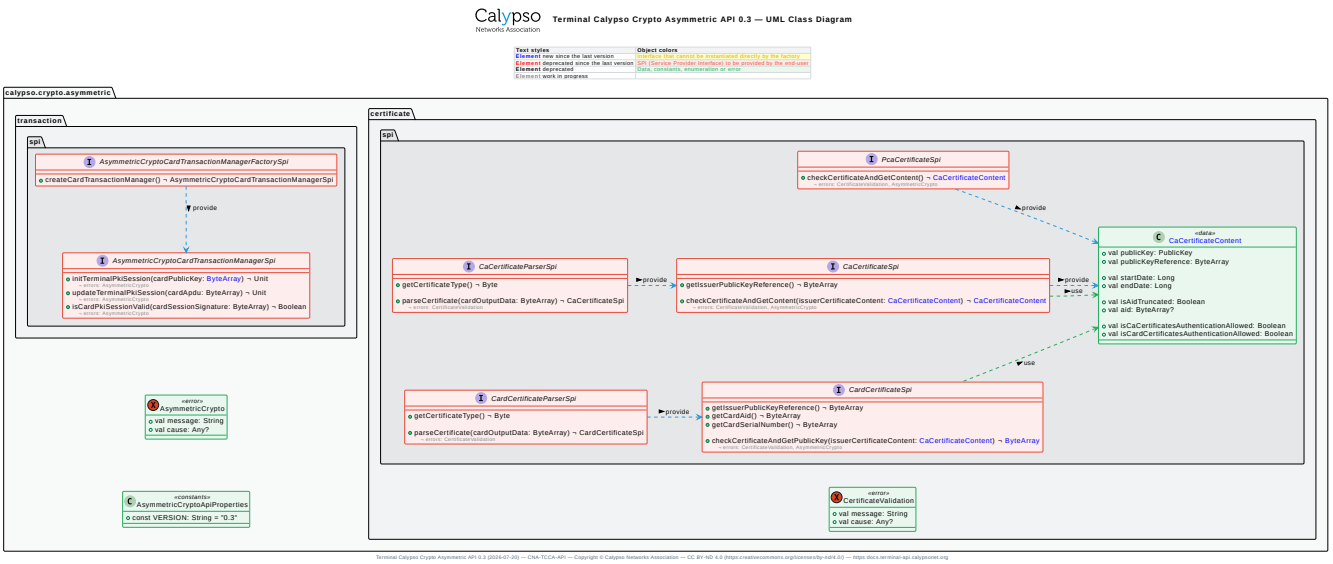


Figure 2. Terminal Calypso Crypto Asymmetric API — UML Class Diagram

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5. API Specification

5.1. Constants

5.1.1. Asymmetric Crypto API Properties

Name	`AsymmetricCryptoApiProperties`
Kind	Constants
Namespace	`calypso.crypto.asymmetric`
Since	0.2
Purpose	Exposes the immutable properties of the Terminal Calypso Crypto Asymmetric API.

Name	Type	Since	Description
`VERSION`	`String`	0.2	String representation of the version of the API implemented by the conforming binding. The value is a dotted decimal of the form `MAJOR.MINOR`; it is `"0.3"` for this revision of the specification.

5.2. Data

5.2.1. CA Certificate Content

Name	`CaCertificateContent`
Kind	Data
Namespace	`calypso.crypto.asymmetric.certificate.spi`
Since	0.2
Purpose	Carries the content of a CA certificate that has been parsed and verified by the crypto module.

Name	Type	Default	Since	Description
`publicKey`	`PublicKey`	—	0.2	Public key carried by the certificate.
`publicKeyReference`	`ByteArray`	—	0.2	Reference of the public key: the stable identifier used by other certificates to designate this key as their issuer. Non-empty.
`startDate`	`Long`	—	0.2	Validity start date of the certificate, formatted `0xYYYYMMDD` (four-digit year, two-digit month, two-digit day). `0` when the validity start date is not defined or available.
`endDate`	`Long`	—	0.2	Validity end date of the certificate, formatted `0xYYYYMMDD`. `0` when the validity end date is not defined or available.
`isAidTruncated`	`Boolean`	—	0.2	Indicates whether the AID carried by the certificate is truncated.
`aid`	`ByteArray?`	—	0.2	AID carried by the certificate, non-empty. `null` when the AID check is not requested; otherwise the AID associated with the certificate MUST be checked.
`isCaCertificatesAuthenticationAllowed`	`Boolean`	—	0.2	Indicates whether the certificate is allowed to authenticate further CA certificates.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Name	Type	Default	Since	Description
`isCardCertificatesAuthenticationAllowed`	`Boolean`	—	0.2	Indicates whether the certificate is allowed to authenticate card certificates.

5.3. SPI Interfaces

Certificate validation follows a **chain**: it MUST start from a PCA certificate (validated through `PcaCertificateSpi.checkCertificateAndGetContent`), traverse zero or more CA certificates, and end on a card certificate that exposes the card's public key. Each `checkCertificateAndGetContent` / `checkCertificateAndGetPublicKey` call MUST validate the signature **and** the metadata (validity dates, AID constraints, certificate purpose) before returning a value.

5.3.1. Asymmetric Crypto Card Transaction Manager Factory SPI

Name	`AsymmetricCryptoCardTransactionManagerFactorySpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.asymmetric.transaction.spi`
Since	0.2
Purpose	Factory of `AsymmetricCryptoCardTransactionManagerSpi`.

Create Card Transaction Manager

Signature	`createCardTransactionManager() → AsymmetricCryptoCardTransactionManagerSpi`
Since	0.2
Description	Creates an `AsymmetricCryptoCardTransactionManagerSpi` providing the cryptographic primitives that a Calypso card transaction requires when using asymmetric (PKI) keys. Implemented by the crypto module.
Returns	An `AsymmetricCryptoCardTransactionManagerSpi`.
Pre-conditions	None.
Errors	None.

5.3.2. Asymmetric Crypto Card Transaction Manager SPI

Name	`AsymmetricCryptoCardTransactionManagerSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.asymmetric.transaction.spi`
Since	0.2
Purpose	Defines the cryptographic primitives required by a Calypso card transaction when using asymmetric keys. An instance is obtained via `AsymmetricCryptoCardTransactionManagerFactorySpi.createCardTransactionManager`.

Init Terminal PKI Session

Signature	`initTerminalPkiSession(cardPublicKey: ByteArray) → Unit`
Since	0.2

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Initialises the cryptographic context for a new PKI secure session with the supplied card public key.
Parameters	`cardPublicKey` — the 64-byte raw value of the card public key, as returned by `CardCertificateSpi.checkCertificateAndGetPublicKey` once the card certificate has been validated.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`AsymmetricCrypto` — if the provided public key value is not compliant with the current elliptic curve, or if an error occurs during the initialisation.

Is Card PKI Session Valid

Signature	`isCardPkiSessionValid(cardSessionSignature: ByteArray) → Boolean`
Since	0.2
Description	Verifies the provided secure session signature against the previously initialised and updated context. This MUST be the final step of the PKI secure session, called after the last `updateTerminalPkiSession`.
Parameters	`cardSessionSignature` — the 64-byte card signature to verify.
Returns	`true` if the signature is valid, `false` otherwise.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`AsymmetricCrypto` — if an error occurs while verifying the signature.

Update Terminal PKI Session

Signature	`updateTerminalPkiSession(cardApdu: ByteArray) → Unit`
Since	0.2
Description	Updates the session signature verification engine with data sent to or received from the card. This operation MUST be called for every APDU exchanged during the PKI secure session, in the order in which it was exchanged. • For ingoing data, the input length MUST be `>= 5`. • For outgoing data, the input length MUST be `>= 2`.
Parameters	`cardApdu` — the APDU bytes exchanged with the card (ingoing or outgoing).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`AsymmetricCrypto` — if an error occurs while updating the session.

5.3.3. CA Certificate Parser SPI

Name	`CaCertificateParserSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.asymmetric.certificate.spi`
Since	0.2
Purpose	Parses CA certificates from raw data stored on a card.

Get Certificate Type

Signature	`getCertificateType() → Byte`
Since	0.2
Description	Returns the certificate type identifier associated with the parser.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Parse Certificate

Signature	`parseCertificate(cardOutputData: ByteArray) → CaCertificateSpi`
Since	0.2
Description	Parses the supplied card output data and returns a new CA certificate instance. The first byte of the input array MUST be the certificate type.
Parameters	`cardOutputData` — a `ByteArray` containing the CA certificate as stored on the card.
Returns	A `CaCertificateSpi`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`CertificateValidation` — if the provided certificate has an unsupported format.

5.3.4. CA Certificate SPI

Name	`CaCertificateSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.asymmetric.certificate.spi`
Since	0.2
Purpose	Represents a Certification Authority (CA) certificate that has been parsed by the crypto module and is ready for chain validation.

Check Certificate And Get Content

Signature	`checkCertificateAndGetContent(`
	`  issuerCertificateContent: CaCertificateContent`
	`) → CaCertificateContent`

Since	0.2
Description	Verifies the certificate signature and other relevant fields, then returns the certificate content. The validation MUST be comprehensive: signature correctness, validity period, issuer and subject details and any relevant constraints or extensions.
Parameters	`issuerCertificateContent` (`CaCertificateContent`) — the issuer certificate content to be used for signature verification.
Returns	A `CaCertificateContent`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`CertificateValidation` — if the certificate is invalid, expired, revoked, or fails any other validation. `AsymmetricCrypto` — if a technical error occurs during the cryptographic computations.

Get Issuer Public Key Reference

Signature	`getIssuerPublicKeyReference() → ByteArray`
Since	0.2
Description	Returns the reference of the issuer's public key, used by the upstream layer to locate the corresponding issuer certificate.
Returns	A non-empty `ByteArray`.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Pre-conditions	None.
Errors	None.

5.3.5. Card Certificate Parser SPI

Name	`CardCertificateParserSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.asymmetric.certificate.spi`
Since	0.2
Purpose	Parses card certificates from raw data stored on a card.

Get Certificate Type

Signature	`getCertificateType() → Byte`
Since	0.2
Description	Returns the certificate type identifier associated with the parser.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Parse Certificate

Signature	`parseCertificate(cardOutputData: ByteArray) → CardCertificateSpi`
Since	0.2
Description	Parses the supplied card output data and returns a new card certificate instance. The first byte of the input MUST be the certificate type. The expected total length is 316 bytes.
Parameters	`cardOutputData` — a `ByteArray` containing the card certificate as stored on the card (316 bytes).
Returns	A `CardCertificateSpi`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`CertificateValidation` — if the provided certificate has an unsupported format.

5.3.6. Card Certificate SPI

Name	`CardCertificateSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.asymmetric.certificate.spi`
Since	0.2
Purpose	Represents a card certificate, used to extract the card's public key after a successful chain validation.

Check Certificate And Get Public Key

Signature	`checkCertificateAndGetPublicKey(`
	`  issuerCertificateContent: CaCertificateContent`
	`) → ByteArray`

Since	0.2

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Verifies the certificate signature and other relevant fields, then returns the card's public key.
Parameters	`issuerCertificateContent` (`CaCertificateContent`) — the issuer certificate content to be used for signature verification.
Returns	A 64-byte `ByteArray` holding the raw value of the card public key.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`CertificateValidation` — if the certificate is invalid. `AsymmetricCrypto` — if a technical error occurs during the cryptographic computations.

Get Card AID

Signature	`getCardAid() → ByteArray`
Since	0.2
Description	Returns the AID of the autonomous application of the card as a `ByteArray` of 5 to 16 bytes.
Returns	A non-empty `ByteArray` of 5 to 16 bytes.
Pre-conditions	None.
Errors	None.

Get Card Serial Number

Signature	`getCardSerialNumber() → ByteArray`
Since	0.2
Description	Returns the serial number of the card as an 8-byte `ByteArray`.
Returns	An 8-byte `ByteArray`.
Pre-conditions	None.
Errors	None.

Get Issuer Public Key Reference

Signature	`getIssuerPublicKeyReference() → ByteArray`
Since	0.2
Description	Returns the reference of the issuer's public key.
Returns	A non-empty `ByteArray`.
Pre-conditions	None.
Errors	None.

5.3.7. PCA Certificate SPI

Name	`PcaCertificateSpi`
Kind	Interface (SPI)
Namespace	`calypso.crypto.asymmetric.certificate.spi`
Since	0.2
Purpose	Represents a Primary Certification Authority (PCA) certificate — the self-signed trust anchor of the Calypso PKI.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Check Certificate And Get Content

Signature	`checkCertificateAndGetContent() → CaCertificateContent`
Since	0.2
Description	Verifies the certificate signature and other relevant fields, then returns the certificate content. The PCA certificate MUST be expected to be self-signed in this context.
Returns	A `CaCertificateContent`.
Pre-conditions	None.
Errors	`CertificateValidation` — if the certificate is invalid, expired, revoked, or fails any other validation. `AsymmetricCrypto` — if a technical error occurs during the cryptographic computations.

5.4. Errors

Every error defined below carries two properties: `message` (`String`), which describes the condition, and `cause` (`Any?`), the underlying error that triggered it, `null` when there is none. They are not restated for each error.

5.4.1. Asymmetric Crypto

Name	`AsymmetricCrypto`
Kind	Error
Namespace	`calypso.crypto.asymmetric`
Since	0.2
Purpose	Indicates that an error occurred when processing an asymmetric cryptographic operation.

5.4.2. Certificate Validation

Name	`CertificateValidation`
Kind	Error
Namespace	`calypso.crypto.asymmetric.certificate`
Since	0.2
Purpose	Indicates an issue encountered during the certificate validation — e.g. an invalid signature or incorrect metadata values.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement